Laboratorio 2

FIFO

```python
# Cola (FIFO) de pasajeros
# La cola es una lista: se agrega al final con append() y se atiende desde
# el inicio con pop(0). Así el primero en entrar es el primero en salir.
cola_pasajeros = []
contador_turnos = 0  # Genera turnos P001, P002, ... que no se repiten.


def registrar_pasajero(raiz):
    """Encola un pasajero al final de la fila (append)."""
    global contador_turnos

    nombre = pedir_texto("Nombre del pasajero: ")
    if nombre is None:
        return

    nombre_destino = pedir_texto("Destino: ")
    if nombre_destino is None:
        return
    destino = obtener_destino_valido(raiz, nombre_destino)
    if destino is None:
        return

    contador_turnos += 1
    pasajero = {
        "turno": f"P{contador_turnos:03d}",
        "nombre": nombre,
        "destino": destino.nombre,
    }
    cola_pasajeros.append(pasajero)
    print(f"Pasajero registrado con turno {pasajero['turno']}.")
```

```python
def atender_pasajero():
    """Desencola al pasajero que llegó primero (pop(0)).

    Se valida antes que la cola no esté vacía: pop(0) sobre una lista
    vacía lanzaría IndexError y detendría el programa.
    """
    if not cola_pasajeros:
        print("No hay pasajeros en espera.")
        return

    pasajero = cola_pasajeros.pop(0)
    print("PASAJERO ATENDIDO")
    print(f"Turno: {pasajero['turno']}")
    print(f"Nombre: {pasajero['nombre']}")
    print(f"Destino: {pasajero['destino']}")


def ver_pasajeros():
    """Muestra la cola en orden de llegada y el total en espera."""
    if not cola_pasajeros:
        print("No hay pasajeros en espera.")
        return

    print("PASAJEROS EN ESPERA")
    for pasajero in cola_pasajeros:
        print(f"{pasajero['turno']} - {pasajero['nombre']} - {pasajero['destino']}")
    print(f"Total en espera: {len(cola_pasajeros)}")


def pasajeros_por_destino():
    """Mejora 2: cuenta cuántos pasaje1ros esperan por cada destino.
```

```python
def pasajeros_por_destino():
    """
    Recorre la cola sin modificarla y acumula los conteos en un diccionario.
    """
    if not cola_pasajeros:
        print("No hay pasajeros en espera.")
        return

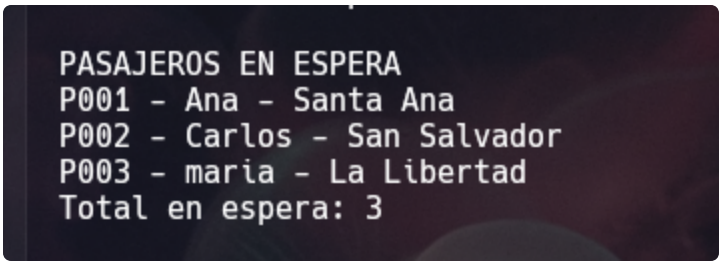
    conteo = {}
    for pasajero in cola_pasajeros:
        destino = pasajero["destino"]
        conteo[destino] = conteo.get(destino, 0) + 1

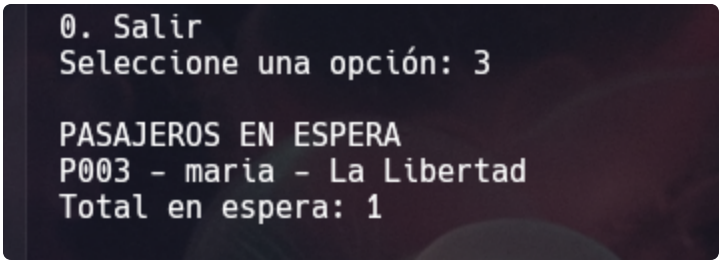
    print("PASAJEROS EN ESPERA POR DESTINO")
    for destino, cantidad in conteo.items():
        print(f"{destino}: {cantidad}")




# A. Pila (LIFO) de salidas

```





LIFO

```python
# Pila (LIFO) de salidas
# La pila es una lista: se apila al final con append() y se desapila desde
# el final con pop(). La última salida registrada es la primera en deshacerse.
pila_salidas = []
contador_salidas = 0  # Genera códigos S001, S002, ...


def registrar_salida(raiz):
    """Apila una nueva salida de bus (append)."""
    global contador_salidas

    codigo_bus = pedir_texto("Código del bus: ")
    if codigo_bus is None:
        return

    nombre_destino = pedir_texto("Destino del bus: ")
    if nombre_destino is None:
        return
    destino = obtener_destino_valido(raiz, nombre_destino)
    if destino is None:
        return

    contador_salidas += 1
    salida = {
        "codigo": f"S{contador_salidas:03d}",
        "bus": codigo_bus,
        "destino": destino.nombre,
        # Mejora 1: se guarda la fecha y hora en que se registró la salida.
        "fecha_hora": datetime.now().strftime("%d/%m/%Y %H:%M:%S"),
    }
```

```python
def registrar_salida(raiz):
    }
    pila_salidas.append(salida)
    print(f"Código del bus: {salida['bus']}")
    print(f"Destino del bus: {salida['destino']}")
    print(f"Salida registrada: {salida['codigo']} ({salida['fecha_hora']})")


def deshacer_salida():
    """Desapila la salida más reciente (pop()).

    Se comprueba primero que la pila no esté vacía para evitar IndexError.
    """
    if not pila_salidas:
        print("No hay salidas registradas para deshacer.")
        return

    salida = pila_salidas.pop()
    print(f"Salida deshecha: {salida['codigo']} - {salida['bus']} - {salida['destino']}")


def ver_salidas():
    """Muestra las salidas desde la base hasta la cima y señala la última."""
    if not pila_salidas:
        print("No hay salidas registradas.")
        return

    print("SALIDAS REGISTRADAS")
    ultima = len(pila_salidas) - 1
    for indice, salida in enumerate(pila_salidas):
        marca = "  <-- última (cima)" if indice == ultima else ""
        print(
```

```python
def ver_salidas():
    print("SALIDAS REGISTRADAS")
    ultima = len(pila_salidas) - 1
    for indice, salida in enumerate(pila_salidas):
        marca = "  <-- última (cima)" if indice == ultima else ""
        print(
            f"{salida['codigo']} - {salida['bus']} - {salida['destino']}"
            f" - {salida['fecha_hora']}{marca}"
        )
    print(f"Total de salidas: {len(pila_salidas)}")
```

```
SALIDAS REGISTRADAS
S001 - 0788 - San Miguel - 15/10/2026 19:47:27
S002 - 0977 - San Salvador - 15/10/2026 19:47:47
S003 - 0977 - La Libertad - 15/10/2026 19:48:02  <-- última (cima)
Total de salidas: 3
```

```
SALIDAS REGISTRADAS
S001 - 0788 - San Miguel - 15/10/2026 19:47:27
S002 - 0977 - San Salvador - 15/10/2026 19:47:47  <-- última (cima)
Total de salidas: 2
```

Arboles

```python
# Árbol de destinos
class NodoDestino:
    """Nodo de un árbol general: guarda un nombre y una lista de hijos.

    A diferencia de un árbol binario (izquierda/derecha), aquí cada nodo
    puede tener cualquier cantidad de hijos, porque una región puede tener
    muchos destinos.
    """

    def __init__(self, nombre):
        self.nombre = nombre
        self.hijos = []

    def agregar_hijo(self, hijo):
        self.hijos.append(hijo)
        return hijo


def construir_arbol_destinos():
    """Crea la jerarquía Terminal Central -> Zonas -> Destinos."""
    raiz = NodoDestino("Terminal Central")

    oriental = raiz.agregar_hijo(NodoDestino("Zona Oriental"))
    oriental.agregar_hijo(NodoDestino("San Miguel"))
    oriental.agregar_hijo(NodoDestino("La Unión"))
    oriental.agregar_hijo(NodoDestino("Usulután"))

    central = raiz.agregar_hijo(NodoDestino("Zona Central"))
    central.agregar_hijo(NodoDestino("San Salvador"))
    central.agregar_hijo(NodoDestino("La Libertad"))
```

```python
def construir_arbol_destinos():
    occidental = raiz.agregar_hijo(NodoDestino("Zona Occidental"))
    occidental.agregar_hijo(NodoDestino("Santa Ana"))
    occidental.agregar_hijo(NodoDestino("Ahuachapán"))

    return raiz


def mostrar_arbol(nodo, nivel=0):
    """Recorrido recursivo en preorden: primero el nodo y luego sus hijos.

    El parámetro nivel controla la sangría, así se visualiza la jerarquía.
    Caso base implícito: un nodo sin hijos no hace más llamadas.
    """
    print("  " * nivel + "- " + nodo.nombre)
    for hijo in nodo.hijos:
        mostrar_arbol(hijo, nivel + 1)


def buscar_destino(nodo, nombre):
    """Búsqueda recursiva. Devuelve el nodo si existe o None si no existe.

    1. Si el nodo actual coincide, se devuelve.
    2. Si no, se busca en cada hijo; en cuanto uno lo encuentra se devuelve.
    3. Si ningún hijo lo encuentra, se devuelve None.
    """
    if normalizar(nodo.nombre) == normalizar(nombre):
        return nodo
    for hijo in nodo.hijos:
        encontrado = buscar_destino(hijo, nombre)
        if encontrado is not None:
            return encontrado
    return None


def obtener_destino_valido(raiz, nombre):
    """Valida que el nombre sea un destino final (hoja) del árbol.

    Se rechazan las regiones y la terminal porque un pasajero o un bus
    viaja a un destino concreto, no a una zona completa.
    """
    nodo = buscar_destino(raiz, nombre)
    if nodo is None:
        print(f"Error: el destino '{nombre}' no existe.")
        return None
    if nodo.hijos:
        print(f"Error: '{nodo.nombre}' es una región, no un destino. Indique un destino específico.")
        return None
    return nodo



# Cola (FIFO) de pasajeros
# La cola es una lista: se agrega al final con append() y se atiende desde
# el inicio con pop(0). Así el primero en entrar es el primero en salir.
cola_pasajeros = []
contador_turnos = 0  # Genera turnos P001, P002, ... que no se repiten.


def registrar_pasajero(raiz):
    """Encola un pasajero al final de la fila (append)."""
    global contador_turnos

    nombre = pedir_texto("Nombre del pasajero: ")
```

```python
# Opciones del árbol en el menú


def opcion_mostrar_arbol(raiz):
    print("ÁRBOL DE DESTINOS")
    mostrar_arbol(raiz)


def opcion_buscar_destino(raiz):
    nombre = pedir_texto("Destino a buscar: ")
    if nombre is None:
        return

    nodo = buscar_destino(raiz, nombre)
    if nodo is None:
        print(f"El destino '{nombre}' no existe.")
    else:
        print(f"Destino encontrado: {nodo.nombre}")
```

```
ÁRBOL DE DESTINOS
- Terminal Central
  - Zona Oriental
    - San Miguel
    - La Unión
    - Usulután
  - Zona Central
    - San Salvador
    - La Libertad
  - Zona Occidental
    - Santa Ana
    - Ahuachapán
```

```
Seleccione una opción: 9

Destino a buscar: san miguel
Destino encontrado: San Miguel
```

Validacion

```
Seleccione una opción: 1

Nombre del pasajero:
Error: el dato no puede estar vacío.
```

```
Destino a buscar: tangamandapio
El destino 'tangamandapio' no existe.
```

Estructura vacia

